

Final Project: Tracing and Verifying AI Agent Tool Calls

Team project: two students per team

1 Project Overview

Build a system that captures, reconstructs, visualizes, and verifies the tool-calling behavior of an AI agent. Use **Hermes Agent** or **OpenClaw** as the agent platform. Your system must show which tool calls actually occurred when an agent answered a question, their order or causal relationships, and their outcomes. The project is an implementation and evaluation task, not merely a diagram of an intended workflow.

Work in a team of exactly two students. Both students are responsible for understanding the entire system, testing it, writing the report, and presenting it. Teams receive the same project score when their documented contributions are substantially equal. The instructor may assign different individual scores when evidence shows unequal contributions.

2 Required Outputs at a Glance

1. Run five required cases: M0–M2 again, plus F1 and F2. Add a run if none shows at least two actual tool calls.
2. Use one documented command to turn each saved run and reference record into a machine-readable graph, a readable visualization, and a validation report.
3. Check every graph against independent execution evidence; show that the validator detects a deliberately altered graph.
4. Submit the technical report, editable report-source link, slides, and reproducible GitHub repository as described below.

3 Learning Objectives

By the end of the project, you should be able to:

1. Explain where tool calls occur in an agent run and what evidence is available to observe them.
2. Design and implement a reproducible trace capture and graph construction method.
3. Distinguish observed call order from inferred causal or data dependencies.
4. Evaluate graph completeness and correctness against an independent source of evidence.
5. Communicate the design, results, limitations, and team contributions in a professional technical report and presentation.

4 Platform and Project Scope

Choose **one** platform: Hermes Agent (Nous Research) or OpenClaw. Record the installed version or commit, model/provider, operating environment, tool configuration, and tracing interface used. Use real agent executions. A hand-written list of expected calls or a simulated graph alone does

not satisfy the implementation requirement. A deterministic mock tool is allowed for a controlled test, but the selected agent must actually invoke it.

Begin with official platform documentation and identify a supported observation point. For example, Hermes documents `pre_tool_call` and `post_tool_call` hooks; OpenClaw documents trajectory bundles, audit records, and OpenTelemetry signals. These are examples, not required implementation choices. Explain the coverage and limitations of the interface you choose. If it omits a type of tool action, supplement it with another observable source and document the reconciliation rule. Official starting points appear in Section 11.

5 Shared Starter Dataset and Questions

Use the synthetic files and questions in `../tooltracking_starter/README.md`. Copy its `data/` folder into your controlled workspace and include the same files in your GitHub repository. **Rerun M0–M2 as new agent runs** using the same question wording as the midterm’s first attempt, with `DATA_DIR` replaced by the current path. Also run **F1 and F2** at least once. F0 and F3 are optional examples. These five required cases cover a no-tool control, simple and cross-file lookups, a three-source investigation, and a missing-file case. Add further runs or your own questions when the required cases do not elicit two actual tool calls or another required behavior. Document all prompt revisions. Compare midterm and final observations where useful, but do not assume the same prompt yields the same calls twice. Validate each final graph against its own independent execution record, not the answer key, the agent’s narrative, or a saved midterm trace.

Illustrative tool-call graphs

Figure 1 shows possible graph shapes for the required tests and a retry. Each box represents a separate *observed* tool invocation, even when the same tool is called again. Arrows in this example mean *next observed call in this run*; they do not claim that one call caused another. The question and final answer can appear in the graph caption or accompanying run record. These are examples only: a real agent may combine file reads into one call, repeat a call, or choose a different order. Your graph must reflect the actual saved run.

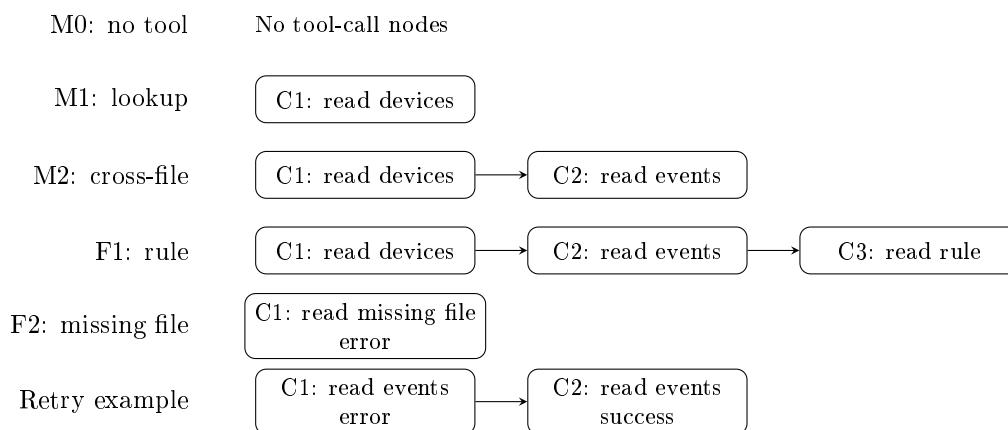


Figure 1: Illustrative call graphs. The retry uses two distinct nodes for the same tool; it is not a required outcome for F1 or F2. A refused F2 request could have no tool-call node.

Fictional branching investigation

Consider the question: *Which device owner, if any, meets alert rule R-1?* Imagine an agent with five example tools: `read_auth_events`, `read_alert_rule`, `read_devices`, `evaluate_rule`, and `match_owner`. These tool names and the run in Figure 2 are fictional teaching examples; they are not built-in Hermes or OpenClaw tools and are not an additional required test.

In this hypothetical run, C1 reads the events, C2 reads the rule, and C3 reads the device records. C4 uses the outputs of C1 and C2 to evaluate the rule. C5 uses the output of C4 and the device records from C3 to identify the owner. Each arrow in Figure 2 means *output used as input*, not next observed call. The three reads could occur in any order; the arrows do not establish parallel execution.

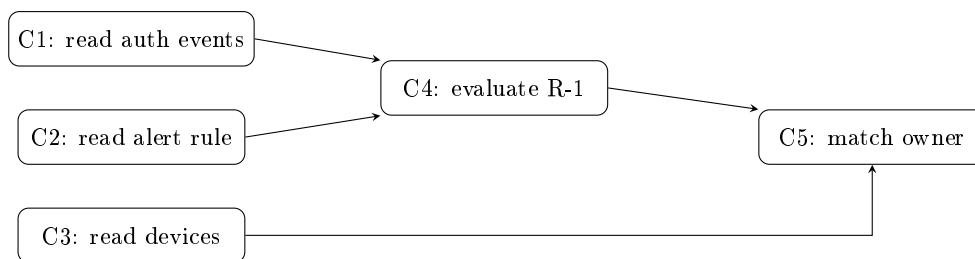


Figure 2: Hypothetical data-dependency graph. Its branching shape is not required of student runs. Show a dependency edge only when evidence establishes that one call’s output was used by another.

This example shows why a complex tool-call graph can have branches and joins. A linear graph is also correct when that is what the observed calls and supported relationships show. Do not add dependency edges merely because a workflow diagram suggests them.

Fictional retry loop and write step

Suppose the first event query returns too few records to decide whether R-1 applies. In an example workflow, an `evaluate_rule` tool reports missing evidence, the agent calls `read_auth_events` again with a wider time window, evaluates the rule again, and then uses a fictional `write_report` tool to save the conclusion in a disposable workspace. These are illustrative tool choices, not required tools or an additional graded case.

Figure 3 distinguishes a *workflow loop* from the graph of actual call instances. The dashed arrow in the top panel means the agent may return to the read step when evidence is incomplete. The lower panel unrolls one possible run: C1 and C3 call the same read tool, but they are different invocations. Its solid arrows mean *next observed call*, as in Figure 1. A real graph must use the observed calls and supported edge relationships; it must not draw a self-loop on one invocation merely because the tool was used again.

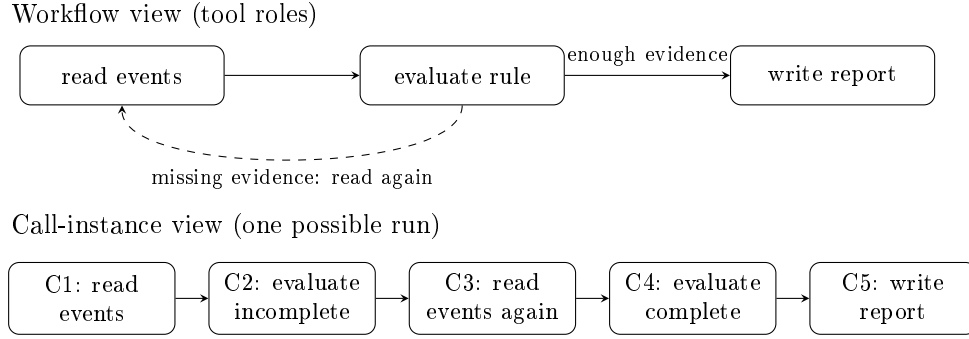


Figure 3: A conceptual workflow loop (top) and one possible sequence of distinct tool-call instances (bottom). The write tool is fictional and is not required for student implementations.

6 Required Technical Work

6.1 Propose a tracing solution

Describe the problem, proposed architecture, alternatives considered, and why you selected your capture method. Include a data-flow diagram from agent event to stored trace, graph, and validation result. Define the trace boundary: one prompt, one run, one session, or another explicit unit. Specify what constitutes a tool call and whether failed, blocked, retried, nested, or parallel calls are included.

6.2 Implement trace capture and graph construction

Implement a runnable collector or parser and a graph generator. Preserve enough metadata to distinguish repeated calls to the same tool: at minimum a run identifier, unique call identifier (native or generated), tool name, event order or timestamp, outcome, and an evidence pointer to the raw record. Capture start/end or request/result events where the platform permits. Record missing fields as unknown; do not invent timestamps, call IDs, or dependencies. If raw arguments or results contain secrets or personal information, redact them in shared artifacts while keeping the evidence needed to verify call identity and order.

Provide a documented command that accepts saved run evidence and its independent reference record and produces, for each run, a machine-readable graph, a readable visualization, and a validation report. The command must work on a new saved run without manually redrawing its graph or editing the code. Automating agent launch and prompt execution is optional; the trace-to-graph-and-validation process is required.

Define graph semantics before generating graphs. At minimum, every observed tool call must appear as a distinct node. Repeated uses of one tool and retries must have separate call IDs and nodes; do not merge them by tool name or draw a self-loop merely because the tool name repeats. Label a call as a retry only when the run evidence supports that relationship. An edge must have a declared meaning, such as *next observed call in a run*, *parent/child call*, *retry of*, or *data dependency*. Do not label temporal adjacency as causation. If your trace supports several edge meanings, use distinct labels or visual styles. Include the user question and final agent response as context in the graph or its caption. Show tool name, call order/ID, and success, error, or blocked status. Save both a machine-readable graph (for example, JSON, GraphML, or DOT) and a readable image or interactive view.

6.3 Demonstrate the actual calling sequence

Run the five required cases in the shared guide: M0–M2, F1, and F2. At least one run must elicit two or more tool calls so that order can be demonstrated. Include a real repeated-call or retry case when you can reproduce one. Otherwise, use a clearly labeled synthetic trace fixture with two calls to the same tool to test that your graph generator keeps both nodes; the fixture does not replace a required real agent run. For each real run, provide the exact question, relevant tool setup, actual trace excerpt, generated graph, validation report, final answer, and a short interpretation of what happened. A prompt that is expected to use a tool but does not is a valid observed result; report it honestly and run additional cases as needed to demonstrate multi-call behavior.

6.4 Verify graph correctness

Establish a **reference record independent of the graph generator**, such as a separate platform audit/trajectory stream, instrumentation around the tool boundary, or a manually annotated raw trace reviewed by both team members. Explain why it can reveal missing or extra calls. For every demonstrated run, compare the generated graph with that record. Check:

1. Node coverage: every actual call appears once; no call is fabricated.
2. Identity and attributes: tool name, call ID, run association, and outcome agree with the record.
3. Edge correctness: each edge follows its declared semantics; ordering claims agree with the evidence. Do not force a total order on truly concurrent calls.
4. Boundary behavior: no-tool, failed/blocked, and repeated or parallel calls are represented correctly where applicable.

For example, if an independent audit lists calls C1 (read devices) and C2 (read events), a graph with only C1 is missing one call. If your edges mean *next observed call*, the graph should also contain C1→C2. This example is a correctness check, not a required call sequence for M2. Report counts of reference calls, graph nodes, matched calls, missing calls, extra calls, correct edges, and incorrect edges for each run. Compute node precision and recall when the reference permits it; define how calls are matched. Explain every discrepancy and fix the implementation or document a justified visibility limit. Demonstrate at least one automated validation test by deliberately removing a known call node or altering an edge in a copy of a graph and showing that the validator detects the change. Human inspection alone is insufficient.

7 Suggested Work Plan

1. Select Hermes or OpenClaw and run a small tool-using example.
2. Read the official tracing documentation and make a capture plan.
3. Define the event schema, graph node/edge semantics, and validation criteria.
4. Implement capture, storage, graph construction, and visualization.
5. Run the required scenarios, save evidence, and verify each graph.
6. Analyze omissions, false edges, failures, overhead, and reproducibility.
7. Prepare the report, presentation, and a live or recorded demonstration.

8 Final Technical Report

Prepare a clear, professional technical report in a standard single-column format. Use an 11- or 12-point readable font, 1-inch margins, numbered section headings, page numbers, figure and table captions, and consistent citations and references. The report may be created in Word, Google Docs,

Overleaf, or another suitable editor, but it must be submitted as a PDF. The main report must be **at least 6 pages**, excluding references and an optional appendix. There is no strict maximum, but the report should remain focused and avoid unnecessary repetition. Include representative screenshots, graphs, and other essential run evidence in the main report; place additional supporting evidence in the optional appendix. Cite official framework documentation and any external code, datasets, or methods. Do not put API keys, private prompts, or sensitive tool outputs in the report source or submitted repository.

Use the following sections and subsections, adapting names only when the same information remains easy to locate:

1. **Title and team members**: state the project title, student names, selected platform, and model access path.
2. **Executive Summary**: summarize the method, main results, and most important limitations.
3. **Introduction**: motivation, problem statement, objectives, and contributions.
4. **Background and Related Work**: agent tool calling, chosen platform, existing trace facilities, and graph interpretation.
5. **Requirements and Design**: trace boundary, event schema, capture point, architecture, graph semantics, privacy decisions, and alternative approaches.
6. **Implementation**: platform/version, configuration, automated trace-to-graph command, generated outputs, reproducibility steps, and the team's GitHub repository URL with the commit hash used for evaluation.
7. **Experimental Method**: M0–M2 reruns and F1–F2, exact prompts, tools, model/settings, test conditions, reference record, matching rules, and metrics.
8. **Results and Verification**: trace/graph examples, per-run comparison table, correctness results, graph-mutation test, discrepancies, and any useful comparison with midterm observations.
9. **Discussion and Limitations**: visibility gaps, concurrency or retries, threats to validity, and improvements.
10. **Conclusion**: what was achieved and what remains unresolved.
11. **References**: consistent citations to all external sources.
12. **Appendix (optional)**: extra graphs, detailed run records, prompts, setup instructions, and AI-assistance disclosure if too long for the main text.

Include a concise **team contributions and AI-assistance statement** in the report or appendix. Identify each student's concrete work and review activities. List the AI systems used, the tasks for which they were used (for example, design ideas, code, test generation, writing, or graph styling), and how the team checked their output. Using ChatGPT, Claude, or similar tools is **strongly recommended throughout the project**; students remain responsible for all submitted claims and code.

9 Deliverables and Submission

Each team must create its **own GitHub repository** for the project source code. Include the repository URL in the final report so a reader can find the exact implementation behind the reported results. Identify the evaluated commit hash there as well. The repository must be accessible to the instructor by the submission deadline.

Each student must independently submit **one Canvas assignment submission** containing:

1. The same final technical report PDF prepared by the team.
2. A share link to the editable report source (for example, Word, Google Docs, or Overleaf); grant the instructor access before submitting. A view-only PDF link is not sufficient.

3. The team’s presentation slides (PPT or PPTX).
4. A link to the team’s GitHub repository containing the runnable trace-to-graph-and-validation command, dependency and setup instructions, redacted saved run evidence, machine-readable graphs, rendered graphs, validation reports and tests, and the prompts/configuration needed to reproduce results. The same URL and evaluated commit hash must appear in the report.
5. A short individual contribution statement, either in Canvas comments or as an attached file, consistent with the team statement in the report.

Both students submit the same team PDF, editable report-source link, slides, and repository link; each supplies their own contribution statement. The **oral presentation is required**. Both students must speak and be prepared to explain the design, code, validation method, and results. Demonstrate at least one question-to-trace-to-graph-to-verification sequence live or with a recorded backup. Follow the instructor’s Canvas announcement for the due date, presentation length, and submission link.

10 Evaluation Rubric (100 Points)

Criterion	Evidence expected	Points
Problem framing and design	Clear objective, platform choice, capture architecture, trace boundary, alternative considered, and justified graph semantics.	10
Automated trace processing	One documented command processes a new saved run and produces graph and validation outputs; unique call identity, accurate status/metadata, and correct handling of repeated, failed, and absent calls.	25
Visualization	Readable per-run graphs; distinct call nodes; meaningful labeled edges; status, question, and answer context; exported machine-readable and visual forms.	20
Testing and demonstration	M0–M2 reruns, F1–F2, a multi-call run, saved evidence, and reproducible execution.	10
Correctness verification	Independent reference; per-run node/edge checks and counts; precision/recall where possible; demonstrated graph-mutation test; explained discrepancies and limitations.	20
Technical report and reproducibility	Required report structure, clear results, consistent citations, setup/dependencies, privacy handling, contribution and AI-use disclosure.	10
Presentation and participation	Both students present, demonstrate one complete run, answer questions, and document substantially equal contributions.	5
Total		100

The rubric awards credit for measured, honest results. A documented visibility gap or failed run can support a strong analysis if the method detects and explains it; an unsupported claim of perfect tracing cannot.

11 References and Official Starting Points

1. Nous Research, “Hermes Agent: Hooks,” <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/hooks.md>.
2. OpenClaw, “Trajectory bundles,” <https://docs.openclaw.ai/tools/trajectory>.
3. OpenClaw, “Audit records,” <https://github.com/openclaw/openclaw/blob/main/docs/cli/audit.md>.
4. OpenClaw, “OpenTelemetry,” <https://github.com/openclaw/openclaw/blob/main/docs/gateway/opentelemetry.md>.

Check the installed platform version against its corresponding documentation; tracing features and event fields can change.